PlacementLens: An Explainable Compiler Pipeline for Heterogeneous AI Graph Placement

Team: Boundary Conditions
SegFault theme: Explainable compilers
Version: 0.1 prototype, September 2026
Repository: https://github.com/Ayush-Kadali/Capstone

Context and status

PlacementLens is the first working slice of our capstone, an explainable hardware-aware runtime for heterogeneous edge-AI execution. The capstone asks how often "offload every supported operator" is the wrong decision on edge devices, and by how much, measured as placement regret against an exhaustively measured oracle.

Status	Components
Built and tested	Graph language and validation, guarded fusion, capability analysis, three placement policies, transfer insertion, lowering to commands, checked runtime, reference evaluator, per-pass explanation records, CLI, verification matrix
First version only	Functional software device (correctness and protocol rules, no timing behaviour); analytical cost model with hand-set constants; per-command host timing; retained-memory accounting without frees
Planned, not built	Calibrated profiles, measured oracle, placement regret, an uncertainty-aware policy, profile-error injection, asynchronous queues, a real edge backend, ONNX import

Problem

An accelerator may support only part of an AI model. Assigning every supported operator to that accelerator can split the graph across backends and add transfers, synchronization, and launch overhead. Operator-level support alone does not explain whether a complete placement is worthwhile. Developers need to see how a capability decision propagates through compiler transformations into executable commands.

What we built

PlacementLens is a dependency-light C++17 compiler and execution prototype for small FP32 matrix graphs. It represents MatMul, ReLU, and Add in a typed graph IR, validates shapes and dependencies, analyzes device capabilities, selects backend placement, inserts transfers, and lowers the graph into explicit allocation, copy, compute, and synchronization commands. A checked synchronous runtime executes those commands with separate host and virtual-device buffers.

The compiler retains each pass's input and output representations, changes, reasons, and downstream consequences. An optional guarded MatMul/ReLU fusion preserves source-node lineage. JSON reports connect source graph, compiled graph, selected placement, lowered commands, modeled costs, and numerical results. A separate host trace records actual command durations.

The native `explain` command produces a readable report on stdout or in Markdown. It accounts for compute, copies, launch and synchronization costs, lists feasible placements, and shows the chosen command stream alongside numerical verification. The `verify` command executes every feasible assignment for a bounded graph.

Pipeline

1. **ValidateGraphPass:** parse a bounded text graph, enforce unique definitions, check references and shapes, reject cycles, and establish a stable topological schedule.
2. **MatMulReluFusionPass, optional:** replace a MatMul followed by a single-use ReLU when the intermediate is not the graph output and the configured target supports ReLU.
3. **CapabilityAnalysisPass:** record legal device assignments and rejection reasons for each operation.
4. **PlacementPass:** choose CPU-only, maximal supported placement, or a bounded modeled-optimal assignment.
5. **InsertTransfersPass:** construct a logical transfer IR using tensor residency and explicit device completion.
6. **LowerToExecutionPlanPass:** materialize storage and emit versioned, sequential commands consumed by the runtime.

This is a custom graph compiler pipeline. It does not integrate LLVM, MLIR, OpenMP, or OpenACC in this version.

Demonstration

The principal example is `MatMul → ReLU → Add`. All plans produce the same result, `[[1,14],[7,1]]`, checked against an independent evaluator of the original graph.

When ReLU is unsupported, maximal placement selects accelerator–CPU–accelerator. Under the declared illustrative cost profile, that plan transfers 112 bytes across six copies and creates two device runs. The modeled-optimal plan uses accelerator–CPU–CPU, transfers 64 bytes across three copies, and creates one device run.

Policy	Placement	Modeled cost, ms	Transfer bytes	Device runs
CPU-only	CCC	9.0000	0	0
Maximal support	ACA	10.7001	112	2
Bounded modeled optimum	ACC	8.1501	64	1

Enabling ReLU support permits a single all-device region. Raising fixed copy cost to 5 ms makes CPU-only the modeled optimum. The report exposes the changed assumption and resulting compiler decisions.

Cost and measurement policy

The placement objective adds per-operation compute constants, a fixed charge plus bytes/bandwidth for each copy, a launch charge per consecutive device run, and explicit synchronization charges. These constants are **illustrative and uncalibrated**, and compute constants do not scale with shape. They are deliberately controllable for explanation.

The virtual device performs real arithmetic in software on the host using separate buffers. Its measured command durations describe this functional host execution; they are not timings of an NPU or GPU. The runtime does not sleep to simulate projected performance. We do not claim measured accelerator speedup or a calibrated profitability predictor.

The cost policy enumerates all capability-valid, memory-feasible assignments for at most 12 graph nodes. It is a bounded optimum of the declared model, not the empirical measured oracle planned for the full capstone.

Correctness and reproducibility

The repository includes a single demo script, explicit tensor inputs, compiler snapshots, command plans, and machine-readable results. Tests cover parsing, topology, shape and datatype errors, capability rejection, guarded fusion, device memory limits, shared-buffer residency, malformed commands, synchronization, and numerical equivalence across placements. Deterministic randomized graphs exercise varying matrix dimensions and values before and after fusion. AddressSanitizer and UndefinedBehaviorSanitizer checks run locally on the exercised paths. See `showcase/VALIDATION.md` for the exact verification record and platform limits.

A reproducible matrix also checks chain, shared-intermediate, and residual-style graphs at widths 2, 8 and 16 under five configurations. The checked run comprises 45 cases, 591 feasible placement executions, and 11,553 commands. These are controlled functional workloads, not 591 application benchmarks or evidence of GPU speedup.

Relation to existing work

ONNX Runtime already uses capability-based execution providers and provides I/O binding to manage device residency. MLIR already supports before/after pass instrumentation. IREE integrates compilation, scheduling and execution across real backends. Our prototype exposes a restricted version of these responsibilities in one reproducible environment, so a user can inspect how a changed assumption propagates from placement to copies and execution. We claim an inspectable implementation and controlled experiments, not a new placement algorithm or a replacement for those systems. Source links and specific comparisons are in showcase/ENGINEERING_NOTES.md in the repository.

Why the Explainable compilers theme

The theme asks teams to make compiler decisions legible to the people using them. Our contribution is that legibility: every pass records its input and output representation, the change, the rule that caused it, and the downstream consequence; command lineage survives fusion; and the plan that is explained is the plan that was executed and numerically checked. The AI graph is the subject matter. The explanation machinery is what we built and what we ask to be evaluated on.

Relation to the official problem statements

The GPU profitability statement (P04) starts from the same observation we do: an offload can be safe and still unprofitable because of transfer, launch and synchronization overheads. It asks for an ML predictor over OpenMP/OpenACC code regions, trained on labelled examples and validated against measured GPU speedups. PlacementLens has no learned model, no source-loop analysis and no GPU measurements, so it is not an implementation of that statement and is not submitted as one.

	P04 predictor	PlacementLens
Unit of decision	One loop or code region, yes/no	Every operator of a graph, jointly
Method	Supervised ML on static and profiled features	Deterministic passes and exhaustive search of a declared analytical model
Output	A prediction with feature-based rationale	An executable command plan with per-pass records
Explanation	Which features drove the model	Which rule fired, what IR changed, what it forced downstream
Evidence	Correlation with measured GPU speedup	Functional correctness of every feasible plan; no speed claim

We share the intent of the pass-analysis statements (P01, P02), per-pass visibility and cascading effects, but operate on our own small graph pipeline without LLVM or MLIR. A learned or calibrated cost model could later replace our constants without changing the passes or the report.

Current boundaries

The frontend accepts a small `.plg` format, FP32 contiguous rank-two tensors, one graph output, and at most 12 operations. Execution is synchronous. Buffers remain allocated until run completion. There is no real-device backend, calibrated hardware cost profile, empirical timing oracle, robust policy, automatic recovery, ONNX importer, or concurrent runtime yet.

Next month

We will first strengthen the graph/plan contracts and reproducible benchmarks, then add measured profiles and an empirical oracle for a bounded workload set. A small real-backend comparison will test which modeled effects transfer to hardware. The paper will explain the foundations, compare relevant compiler/runtime approaches, describe the implemented architecture and methodology, and separate verified results from planned extensions. Concurrency and a richer explorer follow when the headless evidence is stable.

Run

From the repository root:

```
bash scripts/run_demo.sh
build/placementlens explain --graph examples/fragmented.plg \
    --report artifacts/segfault/run-report.md
build/placementlens verify --graph examples/branched.plg --relu-supported
python3 scripts/show_passes.py artifacts/segfault/fragmented.json \
    --policy maximal --pass-name InsertTransfersPass
```

Requirements: C++17 compiler, CMake 3.16+, Python 3 for full checks. No GPU or model download required. The current verified platform is macOS arm64 with AppleClang 17.